

AIopsStressBench: An Offline Stress-to-Decision Benchmark for Operational Time-Series Forecasting

Sicheng Han, Xinyue Chen, Dongyi Lv, Tianyang Shen, Jianhua Mao, Ting Wei, and Zhi Wang*

School of Software Engineering, Xi'an Jiaotong University, Xi'an, China

Email: zhiwang@xjtu.edu.cn

*Corresponding author

Abstract—Time-series forecasting is widely used in cloud operations for capacity planning, alerting, and risk mitigation, but standard benchmarks usually evaluate models on clean, synchronized, offline data. Operational telemetry can be missing, delayed, partially unavailable, shifted by workload changes, and constrained by latency and memory budgets. We present AIopsStressBench, a reproducible offline stress-to-decision benchmark for operational time-series forecasting. It combines public CloudOps/AIops telemetry, controlled telemetry-stress operators, latency and memory profiling, severity analysis, and a forecast-to-capacity open-loop decision proxy. Rather than asking only which model minimizes clean MSE, the benchmark tests whether model rankings remain stable when telemetry health, latency budgets, and asymmetric decision costs change. Across two multivariate operational traces, we find that the clean-MSE winner, the lowest-latency learned model, and the lowest decision-cost model often differ. Patch-based context modeling improves robustness under metric-channel outage, but this advantage is stress-family specific and incurs higher inference cost. These results show that operational forecasting requires stress-to-decision evaluation in addition to clean-error leaderboards.

Index Terms—AIops, time-series forecasting, deployment stress, robustness, latency, CloudOps

I. INTRODUCTION

Cloud operations increasingly rely on time-series forecasting to support resource planning, autoscaling, alerting, and operational risk control. Forecasts of CPU utilization, memory pressure, network traffic, disk IO, and operational KPIs can determine whether a system scales early enough before a workload spike, avoids unnecessary over-provisioning, or detects abnormal behavior before users are affected. Production resource-management and autoscaling systems further motivate this connection between prediction, resource risk, and operational cost [?], [?], [?], [?]. In this setting, forecasting errors have direct operational meaning: underestimation can translate into resource starvation and risk, while overestimation can lead to persistent waste.

Most long-term time-series forecasting studies evaluate models under a clean offline protocol, even when they introduce stronger backbones such as linear decomposition, patch-based Transformers, inverted attention, or multiscale mixing [?], [?], [?], [?]. Forecasting archives and benchmarks such as M4, Monash, GIFT-Eval, and fev-bench broaden reproducible comparison across domains and model families [?], [?], [?], [?], while recent CloudOps and observability datasets highlight

the heterogeneity of operational telemetry [?], [?], [?], [?], [?]. However, clean leaderboards usually assume that the input window is available, synchronized, and drawn from a stable distribution. This assumption is too optimistic for CloudOps telemetry. Monitoring pipelines can drop individual points, lose entire metric channels, delay recent samples, or observe sudden level shifts after releases, migrations, workload changes, and mitigation actions [?], [?], [?]. At the same time, online forecasting can be constrained by P95 latency, memory footprint, and update cost.

This leaves an applied evaluation gap. A practitioner choosing a forecasting model for monitoring or planning needs more than a clean MSE score: the selected model should remain interpretable under missing telemetry, metric-channel outage, stale context, workload shift, latency limits, memory limits, and asymmetric under- versus over-provisioning costs. These factors are usually evaluated separately, if at all, so a clean leaderboard can hide the fact that different operational objectives select different models.

This paper studies AIops forecasting under controlled offline deployment-stress. We build AIopsStressBench, a reproducible evaluation pipeline that combines public operational datasets, chronological NPZ tasks, controlled telemetry-stress operators, supervised and official backbone baselines, latency and memory profiling, a forecast-to-capacity open-loop decision proxy, and case-study windows. The benchmark turns the model-selection question from “which model wins a clean leaderboard?” into “which model remains preferable when telemetry health, latency budget, and asymmetric decision cost change?” The result is a reproducible deployment-stress selection layer for CloudOps forecasting and an accuracy–robustness–latency–decision analysis over public operational telemetry.

Our contributions are:

- We introduce AIopsStressBench, a reproducible offline benchmark artifact for operational forecasting that combines public telemetry traces, fixed chronological splits, controlled telemetry-stress operators, and table/figure generation scripts.
- We define a stress-to-decision evaluation protocol that jointly reports forecast accuracy, stress robustness, P95 latency, memory, and a forecast-to-capacity open-loop decision proxy.

- We provide cross-source model-selection evidence showing that the clean-MSE winner, the lowest-latency learned model, and the lowest decision-cost model can differ under the same operational telemetry protocol.
- We release a reviewer-auditable artifact with public-data conversion scripts, NPZ manifests, stress operators, model wrappers, table/figure generation scripts, and a minimal path for reproducing the main evidence chain.

II. RELATED WORK

Long-term forecasting backbones and benchmarks. Recent long-term forecasting work has improved accuracy through simple decomposition, patch tokenization, channel-wise inversion, multiscale mixing, and temporal variation modeling [?], [?], [?], [?], [?]. Forecasting benchmarks and archives such as M4, Monash, GIFT-Eval, and fev-bench broaden empirical comparison, reproducible infrastructure, covariate coverage, and aggregation rigor across domains and models [?], [?], [?], [?]. These efforts are important context, but their standard clean-data protocols do not directly answer whether a model remains deployable when telemetry is missing, delayed, shifted, or latency constrained.

Foundation models for time series. Foundation models such as Chronos, Moirai, and TimesFM study pretraining and zero-shot forecasting across diverse time-series corpora [?], [?], [?]. Recent efficiency studies further argue that time-series foundation models should be evaluated with runtime or energy cost, not only accuracy [?]. They are relevant to CloudOps because operational telemetry can be heterogeneous and costly to label or tune. We therefore include a bounded Chronos-Bolt zero-shot reference on the two multivariate sources to report feasibility, latency, and stress sensitivity, while the main empirical claims focus on reproducible supervised and official backbone comparisons under the native stress protocol.

AIops telemetry, autoscaling, and public traces. CloudOps forecasting has recently received more attention through large-scale pretraining datasets such as the Salesforce CloudOps collection [?]. Observability-native benchmarks such as BOOM show that production observability metrics have heavy-tailed, heterogeneous structure and deserve domain-specific evaluation [?]. Public AIops and cluster traces also support reproducible operational studies, including GAIA [?], NetMan KPI datasets [?], Alibaba Cluster Trace 2018 [?], and Borg-style workload traces inside the Salesforce CloudOps release. Autoscaling and resource-management systems such as Autopilot, Sinan, and AHPA motivate why forecast quality should be connected to operational cost and resource risk [?], [?], [?]. Monitoring systems also expose practical telemetry concerns: missing data handling, metric freshness, stale series, and unavailable pod metrics are explicit operational cases in production monitoring and autoscaling documentation [?], [?], [?], [?]. These sources and systems differ in granularity and application meaning; GAIA and NetMan are strong for KPI stress diversity, while Alibaba and Salesforce/Borg better support multivariate operational telemetry analysis.

TABLE I
BENCHMARK POSITIONING. AIOPSSTRESSBENCH COMBINES CLOUDOPS TELEMETRY, CONTROLLED STRESS, LATENCY/MEMORY PROFILING, DECISION-PROXY ANALYSIS, AND MODEL-SELECTION FOCUS.

Benchmark	Ops tel.	Stress	Lat./mem.	Decision	Selection
M4/Monash/GIFT	–	–	–	–	–
fev-bench	–	–	–	–	–
FM cost benchmarks	–	–	Y	–	–
BOOM observability	Y	–	–	–	–
WOODS/OOD	–	Y	–	–	–
CloudOps/AIops data	Y	–	–	–	–
AIopsStressBench	Y	Y	Y	Y	Y

Robust and deployment-aware evaluation. Missing-value models and imputation methods such as GRU-D and BRITS show that incomplete time series need explicit treatment [?], [?]. Drift and out-of-distribution studies further show that clean validation can fail when the test distribution changes [?], [?]. Robustness benchmarks such as WOODS study distribution shift, while observability benchmarks such as BOOM characterize production telemetry structure. AIopsStressBench targets the gap between these lines: it does not compete on archive scale, but couples public operational telemetry, controlled telemetry-degradation operators, latency/memory profiling, a forecast-to-capacity open-loop decision proxy, and model-selection analysis in one reproducible stress-to-decision protocol.

Existing clean forecasting, robustness, observability, and foundation-model benchmarks answer important but different questions. Table ?? summarizes this positioning. AIopsStressBench narrows the target to a joint stress-to-decision question: under the same public operational trace, do telemetry degradation, latency budget, and asymmetric decision cost change which forecaster a practitioner would select?

III. PROBLEM DEFINITION

Let an operational trace be a multivariate tensor $X \in \mathbb{R}^{E \times T \times M}$, where E is the number of entities, T is time, and M is the number of telemetry metrics. From X we construct chronological forecasting windows $(\mathbf{x}_t, \mathbf{y}_t)$ with input length $L = 96$ and horizon $H = 24$, where $\mathbf{x}_t \in \mathbb{R}^{E \times L \times M}$ is the recent context and $\mathbf{y}_t \in \mathbb{R}^{E \times H}$ is the target metric trajectory. A forecasting model f_θ produces $\hat{\mathbf{y}}_t = f_\theta(\mathbf{x}_t)$.

A clean forecasting benchmark evaluates f_θ on unmodified inputs. AIopsStressBench instead evaluates the stressed input $\tilde{\mathbf{x}}_t = s_\phi(\mathbf{x}_t)$, where s_ϕ is a controlled telemetry-stress operator such as point missingness, metric-channel outage, delayed telemetry, noise, burst, or level shift. The target $\mathbf{y}_t$ remains clean, so the benchmark measures how input-side telemetry degradation changes forecasting and downstream model selection rather than changing the prediction target itself.

The selection problem is multi-objective. For each model and stress setting, we report accuracy $A(f_\theta, s_\phi)$, robustness degradation $R(f_\theta, s_\phi)$, P95 inference latency $\ell_{95}(f_\theta)$, memory

TABLE II
AIOpsSTRESSBENCH BENCHMARK CARD.

Component	Setting	Details
Task	Operational forecasting under deployment stress	Input length 96, horizon 24, chronological 65%/15%/20 split.
Public sources	GAIA, NetMan, Alibaba 2018, Salesforce/Borg 2011	Alibaba and Salesforce/Borg are multivariate telemetry; GAIA/NetMan provide KPI stress diversity.
Stress operators	Missing points, metric outage, delayed tail, noise, burst, level shift	Stress is injected only into the input window; targets remain clean.
Metrics	MSE/MAE plus deployment metrics	P95 latency, parameters, memory, severity slope, and forecast-to-capacity proxy.
Core comparable pool	DLinear, RACE-DLinear, PatchTST-lite, official PatchTST, native iTransformer, TimeMixer	Native or native-wrapper stress protocol where available; RACE-DLinear is a lightweight stress-aware baseline used for robustness analysis.
Reference-only pool	LastValue, Chronos-Bolt zero-shot, LITSF-bridge iTransformer	Context, lower-bound, and feasibility references; these rows do not drive the central model-ranking claims.
Reproducibility	Minimal and extended reproduction paths	Minimal path regenerates the core winner and latency tables; extended path regenerates severity, sensitivity, and multi-seed results.

$m(f_\theta)$, and a forecast-to-capacity open-loop decision proxy $C(f_\theta, s_\phi)$. The operational question is therefore not only

$$\arg \min_{f_\theta} A(f_\theta, \text{clean}), \quad (1)$$

but whether the preferred model changes when the selection criterion includes telemetry stress, latency or memory constraints, and asymmetric decision cost. This formalizes three common paths: low-latency alerting, batch capacity planning, and degraded-telemetry response.

IV. EVALUATION PIPELINE

Figure ?? gives the pipeline overview, and Table ?? summarizes the benchmark card. The evaluation has six stages. First, each dataset is audited and converted to a tensor with shape [entities, time, metrics]. Second, a rolling-window forecasting task is built with input length 96 and prediction horizon 24. Third, deployment stress is injected into the input window. Fourth, each model is evaluated by forecasting accuracy, latency, memory, and parameters. Fifth, predictions are passed to a forecast-to-capacity open-loop decision proxy. Sixth, selected windows are visualized as case studies.

A. Deployment Stress Protocol

All experiments use input length 96, prediction horizon 24, seed 42, and chronological train/validation/test splits of 65%/15%/20%. Stress is applied only to the input window at train or evaluation time; the forecasting target remains clean, so the protocol measures whether a model can make an operationally useful forecast from degraded telemetry.

We evaluate the six deployment stresses summarized in Table ?. Missing points randomly mask individual input values. Missing variables mask entire metric channels within an input window, simulating telemetry outage. Delayed tail masks the most recent 12 input steps, simulating late-arriving monitoring data. Noisy stress adds Gaussian input noise with standard deviation 0.2. Burst stress injects sparse positive spikes at rate 0.02. Level shift adds a constant shift of 0.4 to the second half of the input window, simulating release, migration, or workload-regime changes. These stresses are not meant to cover every production failure mode; they provide a reproducible minimum protocol for deployment-aware forecasting.

TABLE III
DEPLOYMENT-STRESS TAXONOMY AND OPERATIONAL MOTIVATION.

Stress	Protocol	Operational source	Evaluation
Missing points	Random input values unavailable	Packet loss, scraper gaps	Robustness curve
Metric outage	Entire input metric channel masked	Exporter failure, telemetry outage	Outage stress
Delayed tail	Most recent k input steps hidden	Ingestion lag, delayed monitoring	Delay stress
Noise	Gaussian perturbation on input	Aggregation or sensor instability	Noise stress
Burst	Sparse positive spikes in input	Workload spike, flash crowd	Burst stress
Level shift	Context level changes before horizon	Release, migration, mitigation	Shift stress

TABLE IV
COMPACT STRESS-CALIBRATION PROXIES FROM PUBLIC TELEMETRY. VALUES MOTIVATE CONTROLLED OPERATORS BUT DO NOT VALIDATE INCIDENT FREQUENCY.

Public-trace proxy	Alibaba	Salesforce/Borg	Stress operator motivated
Zero-run ratio	0.0328	0.1886	Metric outage
Flatline windows	0.625	0.654	Stale telemetry
Robust spike rate	0.0062	0.0231	Burst/noise
P95 level-shift score	1.7032	1.3717	Level shift

B. Stress Realism Audit

Public operational traces are often cleaned before release and underrepresent monitoring failures, so we treat stress realism as calibration rather than validation. This calibration uses two complementary signals. First, as shown in Table ??, we audit compact public-trace proxies such as non-finite values, zero rates, long zero runs, flatline windows, robust spikes, and level-shift scores. Second, a public RE2-OB fault-injection slice provides external sanity evidence about whether externally provided fault windows can expose the same type of ranking disagreement. Together, these sources motivate stress levels and boundary conditions, but they do not estimate operator-specific incident frequency or uniquely identify root causes such as exporter failure or ingestion delay. Metric outage has the clearest public-trace proxy support in Alibaba and Salesforce/Borg, while point missingness and tail delay remain controlled probes for reproducible sensitivity analysis.

The public RE2-OB fault-injection slice is evaluated without additional injected corruption and serves as complementary sanity evidence rather than incident validation; the 58-case compact summary is retained in the artifact because the target is Istio P99 latency rather than resource demand.

V. DATASETS

GAIA metric forecast [?] contains operational KPI series with categories such as periodic, changepoint, low-SNR, and partially-stationary behavior. It is useful for stress diversity, but it is mostly single-target KPI data.

NetMan KPI [?] provides another public operational KPI source and supports missing-telemetry case studies. Like GAIA, it should not be overused for resource-capacity claims.

Alibaba Cluster Trace 2018 [?] machine usage is the first multivariate source. We use machine-level CPU utilization, memory utilization, network in/out, and disk IO. It supports a resource-telemetry story, but it should not be described as service-level request telemetry.

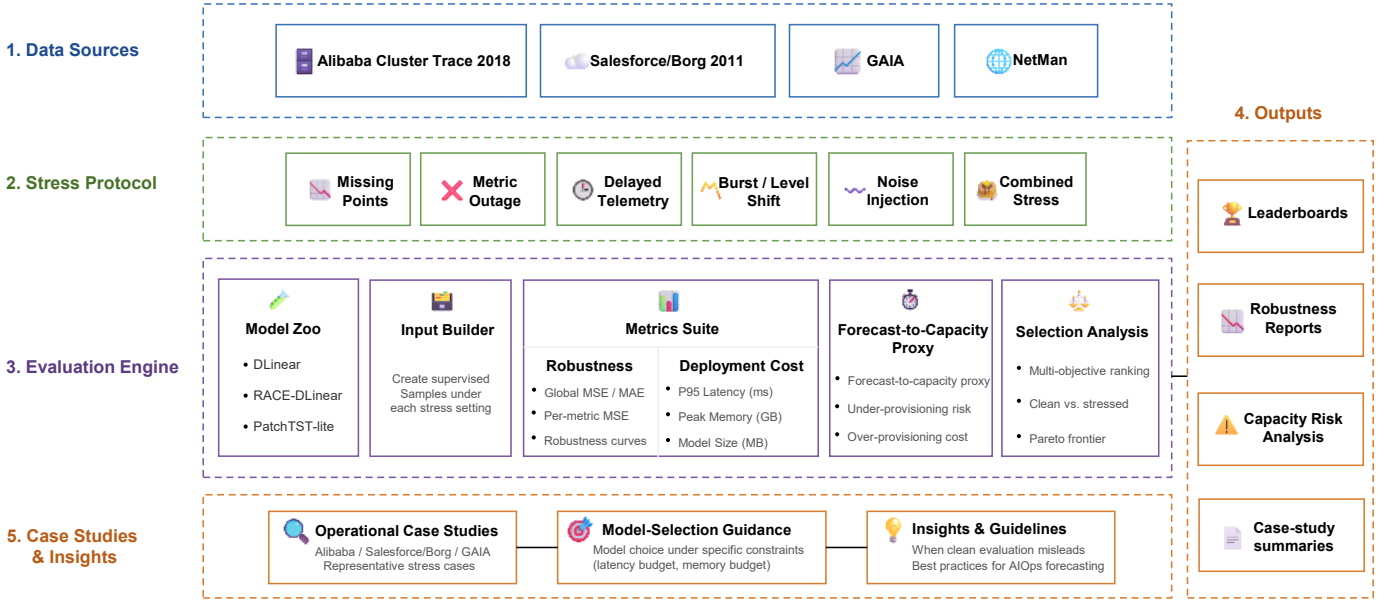


Fig. 1. AIOPSStressBench pipeline. Public telemetry is audited, windowed, evaluated under controlled offline deployment-stress operators, profiled for latency and memory, and connected to within-protocol decision-proxy analysis and case studies.

Salesforce CloudOps/Borg 2011 [?] is the second multivariate operational source. We convert the public Salesforce CloudOps TSF Borg split into a tensor with 256 entities, 2048 time steps, and 7 dynamic metrics. One caveat is that `dynamic_4` is all zero in the current subset; we keep it in the released tensor for source-schema reproducibility and audit transparency, but exclude it from metric-level interpretation. This dataset strengthens the multi-source evidence because the same native stress suite, official baselines, severity curves, and multi-seed checks are reproduced beyond Alibaba.

We use the datasets for different evidential roles. GAIA and NetMan support KPI stress diversity and degraded-telemetry case studies. Alibaba and Salesforce/Borg provide the main multivariate resource-telemetry evidence for latency, stress, and capacity-proxy analysis. Capacity-related conclusions are therefore reported only for resource-level telemetry sources and should not be extrapolated from KPI-only datasets.

VI. BASELINES

Native pipeline baselines include last-value, DLinear [?], RACE-DLinear, and a lightweight patch-based model. We use PatchTST-lite to denote our lightweight native patch-based implementation evaluated inside the unified NPZ/stress pipeline, and Official PatchTST [?] to denote the imported official backbone evaluated through a native wrapper. RACE-DLinear is a lightweight robust baseline used to test whether simple missing-aware linear modeling can improve deployment-stress behavior.

Official PatchTST is integrated by importing the official model class and evaluating it inside the native NPZ/stress pipeline. This keeps missing-variable stress and decision-proxy evaluation comparable with native baselines. Official iTransformer and TimeMixer [?], [?] are evaluated with native

wrappers on the two multivariate telemetry sources for the core deployment stresses. Chronos-Bolt [?] is evaluated only as a zero-shot target-metric reference on a bounded subset, not as a full foundation-model benchmark. Older LTSF-bridge iTransformer runs are retained only as reference analysis because they flatten entity-metric channels and are not perfectly equivalent to the native per-entity target protocol.

Baseline comparability is enforced by the native NPZ windows, the same stress operators, the same target metric, and the same latency profiler; Table ?? summarizes the model pools. All supervised learned models are trained on the same clean chronological training windows unless explicitly marked as stress-augmented; all stress results are test-time input corruptions with clean targets. RACE-DLinear is the stress-aware/training-stress baseline and is not positioned as a leading-model claim. Official wrappers are comparable references, not members of the lightweight five-seed pool. Where not otherwise stated, native and wrapped backbones follow the released settings from the original papers or official implementations after adapting input/output dimensions to the unified NPZ/stress pipeline; the released baseline-fairness card records the main implementation notes and core hyperparameter choices. The main winner tables exclude LastValue, Chronos-Bolt, and LTSF-bridge rows from learned-model selection.

VII. METRICS

We report MSE and MAE, but they are not sufficient for the Applied story. We also report P95 latency, parameter count, GPU memory, and a forecast-to-capacity open-loop decision proxy. Native latency profiling uses a forward-only stressed input batch of at most 32 windows, 8 warmup iterations, 40 timed iterations, and `torch.cuda.synchronize()`

TABLE V
COMPACT BASELINE-POOL COMPARABILITY CARD FOR INTERPRETING
WINNER TABLES.

Pool	Models	Training/protocol	Used for
Lightweight core	DLinear; RACE-DLinear; PatchTST-lite	Same clean chronological windows; RACE marked stress-aware	Five-seed claims
Official/native wrappers	Official PatchTST; Native iTransformer; Official TimeMixer	Same NPZ/stress protocol where available	Coverage and single-run winner tables
References	LastValue; Chronos-Bolt; LTSF bridge	Not comparable supervised training	Context only; excluded from winner selection

around CUDA timing; it excludes data loading and normalization. GPU memory is peak allocated memory from `torch.cuda.max_memory_allocated`. Native runs were observed on NVIDIA RTX 4090-class hardware, while the bounded Chronos-Bolt reference is reported separately from A6000 runs.

The forecast-to-capacity open-loop decision proxy compares predicted capacity with true demand under a headroom factor. Models may train and predict on standardized tensors, but decision costs are computed after inverse transformation on the original non-negative resource scale. For a normalized forecast $\hat{z}_t$, let $\hat{y}_t^{raw} = \text{inv_norm}(\hat{z}_t)$ and let y_t^{raw} be the raw target demand. The proxy provisions

$$c_t = \max(\hat{y}_t^{raw}, 0)(1 + h), \quad (2)$$

then computes

$$\text{cost}_t = c_u \frac{\max(y_t^{raw} - c_t, 0)}{\max(y_t^{raw}, \epsilon)} + c_o \frac{\max(c_t - y_t^{raw}, 0)}{\max(y_t^{raw}, \epsilon)}. \quad (3)$$

This metric is a reproducible forecast-to-capacity open-loop decision proxy. It intentionally excludes queueing, cold starts, scheduler constraints, service dependencies, closed-loop control delay, and service-level SLOs, and it does not apply feedback correction within the forecast horizon. We use the proxy for one narrow purpose: to test whether forecasts with similar MSE can imply different under- and over-provisioning consequences under a fixed asymmetric-cost protocol. The default $c_u : c_o = 5 : 1$ encodes the operational assumption that resource starvation or missed capacity is more costly than short-lived over-provisioning waste; Table ?? reports compact sensitivity checks.

Unless otherwise noted, we use headroom $h = 0.15$, under-provision cost $c_u = 5.0$, over-provision cost $c_o = 1.0$, and demand floor $\epsilon = 0.05$. The target is metric index 0 for each dataset: `cpu_util_percent` for Alibaba and `target_0` for the Salesforce/Borg public subset, treated as a non-negative resource-demand proxy. Costs are computed per entity, window, horizon step, and target metric, then averaged over the evaluated test windows. The same target metric is used for forecast accuracy and the capacity proxy.

A. Forecast-to-Capacity Decision Proxy

To make the Applied evaluation less dependent on MSE and MAE, we add a forecast-to-capacity open-loop decision

TABLE VI
CROSS-SOURCE SCENARIO WINNERS IN THE SINGLE-RUN ALL-MODEL POOL. THE TABLE TESTS WHETHER BEST-MSE, LOWEST-P95 LEARNED, AND BEST DECISION-COST MODELS AGREE UNDER THE SAME SOURCE AND STRESS SETTING. LASTVALUE, CHRONOS-BOLT, AND LTSF-BRIDGE ARE EXCLUDED FROM WINNER SELECTION; MULTI-SEED STABILITY IS REPORTED IN TABLE ??.

Source	Stress	Pool	Best MSE	Best P95	Best decision	Flip
Alibaba	Clean	all learned	RACE-DLinear	DLinear	Native iTransformer	Y
Alibaba	Delayed tail	all learned	RACE-DLinear	DLinear	Official PatchTST	Y
Alibaba	Missing points	all learned	DLinear	DLinear	Official PatchTST	Y
Alibaba	Metric outage	all learned	PatchTST-lite	DLinear	PatchTST-lite	Y
Salesforce/Borg	Clean	all learned	Official TimeMixer	Official PatchTST	Official TimeMixer	Y
Salesforce/Borg	Delayed tail	all learned	Official TimeMixer	Native iTransformer	Official PatchTST	Y
Salesforce/Borg	Missing points	all learned	Official TimeMixer	Native iTransformer	Official PatchTST	Y
Salesforce/Borg	Metric outage	all learned	Native iTransformer	Native iTransformer	Native iTransformer	N

proxy. A forecast policy provisions $\max(\hat{y}, 0)(1 + h)$ capacity for the future horizon. We report Reactive HPA as a controller reference; it uses the previous realized demand as a one-step delayed controller and helps separate online reaction from proactive capacity planning. The proxy reports under-provision area, over-provision area, peak-miss rate, severe-under rate, P95 under-provision ratio, and total normalized cost. Reactive HPA is reported as a controller reference rather than as a member of the clean/stress forecasting winner pool.

VIII. RESULTS

We organize the results around three claims. Claim 1 asks whether clean accuracy is sufficient for within-protocol model selection. Claim 2 tests whether robustness depends on the stress family. Claim 3 connects benchmark evidence to deployment constraints such as latency budgets and decision-proxy objectives.

The results are organized as an evidence chain rather than as a raw leaderboard. Claim 1 is anchored by the cross-source winner table, which tests whether clean-MSE, lowest-latency, and decision-cost winners agree on Alibaba and Salesforce/Borg. Claim 2 is supported by stress-family and severity analyses, which test whether robustness is operator-specific. Claim 3 is supported by latency, memory, decision-proxy, and sensitivity tables, which test whether deployment constraints change the selected model. We use the all-model single-run pool to test coverage across modern backbones, and the lightweight five-seed pool to test statistical stability under the fully repeated setting. Extended sweeps and auxiliary ablations are part of the reproducibility package.

A. Claim 1: Clean Accuracy Is Insufficient

Takeaway: clean accuracy is necessary for filtering models, but it is insufficient for selecting an operational forecaster under latency and decision-proxy constraints.

Table ?? is the first result table because it shows the model-selection problem across both multivariate sources. In most compact source-stress settings, the best clean/stressed MSE model, the lowest-P95 learned model, and the lowest decision-cost model are not all the same. Figure ??(d) uses the same eight compact settings as its denominator. Salesforce/Borg is not only a supporting appendix row: it reproduces the same objective-disagreement pattern under clean, delayed-tail, and missing-point settings, while metric outage selects native

TABLE VII
CLEAN ALIBABA FORECASTING ACCURACY AND DEPLOYMENT COST.

Model	MSE	MAE	P95 ms	Params	Mem MB
RACE-DLinear	0.2671	0.3783	0.355	4668.0	17.00
DLinear	0.2697	0.3837	0.1369	2334.0	17.00
Native iTransformer	0.2744	0.3744	1.9605	2.81e+05	30.00
Official PatchTST	0.2748	0.3764	2.1504	3.06e+05	155.0
PatchTST-lite	0.2753	0.3845	0.6482	1.24e+05	37.00
Official TimeMixer	0.277	0.3799	4.9689	1.29e+05	178
Chronos-Bolt ref.	0.3922	0.4705	10.95	pretrained	26.00
LastValue	0.4082	0.4475	0.0734	0	0

iTransformer across all three objectives in the single-run all-model pool. The point is not that one model always wins; it is that the selected model depends on the operational objective.

Table ?? then gives the detailed Alibaba clean row behind the compact winner summary. RACE-DLinear obtains the lowest clean MSE, but its advantage over DLinear is small. DLinear is slightly less accurate, yet it is the lowest-latency learned model. PatchTST-lite, Official PatchTST, native iTransformer, and official TimeMixer are close in clean accuracy, but they require higher inference cost. The TimeMixer result is useful because it broadens the model pool without changing the deployment conclusion: a stronger modern backbone is not automatically the right online model when latency is a hard constraint.

This result is already a warning for within-protocol model selection. A clean leaderboard would favor the best MSE row, but an online monitoring path with a strict P95 latency budget may reasonably select a faster learned model, while an offline planning job may accept a heavier model if it lowers decision cost. Clean accuracy is therefore a necessary metric, not a sufficient Applied decision rule. The main evidence for this claim is Table ??, the Alibaba detail in Table ??, and the five-seed stability check in Table ??.

B. Claim 2: Robustness Is Stress-Family Specific

Takeaway: robustness is stress-family specific; patch-based context helps most under metric outage, but this advantage does not automatically transfer to every telemetry corruption.

Table ?? evaluates the same models after deployment stress is injected into the input window. Among native stress-suite models, PatchTST-lite has the best mean stress MSE and worst-case MSE, suggesting that patch-based context modeling can improve robustness under corrupted telemetry. DLinear remains much faster but loses robustness under severe stress. RACE-DLinear slightly improves over DLinear in mean and worst stress MSE, but the margin is not large enough to support a new-model claim.

Official PatchTST, native iTransformer, and official TimeMixer are evaluated on the same Alibaba native stress suite for core deployment stresses. They are competitive on several clean or mild-corruption scenarios but remain heavier than the native lightweight baselines, and metric outage is still difficult. TimeMixer broadens the model pool through multi-scale mixing, but the current native run does not remove the need for channel-outage-specific robustness: losing an entire

TABLE VIII
ALIBABA DEPLOYMENT-STRESS ROBUSTNESS.

Model	Mean MSE	Mean MAE	Worst MSE	P95 ms	Mem MB
PatchTST-lite	0.2981	0.4044	0.3486	0.665	37.00
RACE-DLinear	0.3086	0.4151	0.4484	0.3899	17.00
DLinear	0.3122	0.4206	0.464	0.1439	17.00
Official PatchTST	0.3178	0.4086	0.5295	1.9425	155.0
Official TimeMixer	0.333	0.4156	0.522	4.6647	179
Native iTransformer	0.3407	0.4207	0.5309	1.911	30.00
Chronos-Bolt ref.	0.4827	0.5139	0.5212	11.71	26.00
LastValue	0.6803	0.601	1.1093	0.0737	0.1429

telemetry stream remains different from modeling multi-scale trends. We also include a bounded Chronos-Bolt zero-shot feasibility reference, reported as additional foundation-model context over a subset of windows. The Applied takeaway is that one pretrained reference does not remove the need for deployment-aware model selection.

The same stress-family pattern appears beyond the Alibaba detail table. On metric outage, DLinear degrades faster than PatchTST-lite on both multivariate sources, with a larger absolute slope on Salesforce/Borg. Under additive noise, the degradation slopes of DLinear, RACE-DLinear, and PatchTST-lite are much closer. The evidence for this claim is Table ??, Figure ??, and the five-seed stability check in Table ??; the practical implication is that “robust model” is not a single global label.

C. Imputation Check: Does Simple Repair Remove the Problem?

Takeaway: common imputation pipelines can reduce point-missing damage, but they do not eliminate deployment-stress evaluation because channel outage and stale context still change the model choice.

Real monitoring pipelines often apply simple repair before forecasting. We therefore add DLinear and PatchTST-lite with forward-fill and mean-fill preprocessing after stress injection. The ground truth remains clean. These rows serve as pipeline baselines for deployment-stress analysis; the full generated imputation table is retained in the artifact.

The pattern is consistent with deployment intuition. On point missingness, simple repair can help: Alibaba DLinear missing-30 capacity cost drops from 0.508 without repair to about 0.414–0.415 with mean or forward-fill. However, whole-channel outage is not solved by filling because the missing channel carries no contemporaneous information in the window; Alibaba DLinear missing-variable capacity cost remains 0.889 across no-fill, forward-fill, and mean-fill. Thus imputation is a useful pipeline baseline, but it does not remove the need to evaluate telemetry outage and stale-context stresses.

D. Claim 3a: Capacity Objectives Change Model Choice

Takeaway: the forecast-to-capacity open-loop decision proxy exposes consequences that are hidden by MSE/MAE alone, especially when under-provisioning and over-provisioning have asymmetric costs.

TABLE IX
ALIBABA DEPLOYMENT COST ACROSS CLEAN AND STRESS RUNS.

Model	Clean P95	Mean P95	Params	Mem MB
LastValue	0.0734	0.0736	0	0.125
DLinear	0.1369	0.143	2334.0	17.00
RACE-DLinear	0.355	0.3855	4668.0	17.00
PatchTST-lite	0.6482	0.6629	1.24e+05	37.00
Native iTransformer	1.9605	1.9193	2.81e+05	30.00
Official PatchTST	2.1504	1.9656	3.06e+05	155.0
Official TimeMixer	4.9689	4.7154	1.29e+05	179
Chronos-Bolt ref.	10.95	11.46	pretrained	26.00

TABLE X
ALIBABA SCENARIO WINNERS WITHIN THE CORE LEARNED COMPARABLE POOL: DLINEAR, RACE-DLINEAR, PATCHTST-LITE, OFFICIAL PATCHTST, NATIVE ITRANSFORMER, AND TIMEMIXER.

Stress	Best MSE	Best learned P95	Best Capacity
clean	RACE-DLinear	DLinear	Native iTransformer
delayed_12	RACE-DLinear	DLinear	Official PatchTST
level_shift	RACE-DLinear	DLinear	Native iTransformer
missing_30	DLinear	DLinear	Official PatchTST
missing_variables_30	PatchTST-lite	DLinear	PatchTST-lite

Table ?? summarizes deployment cost on Alibaba. LastValue is a useful zero-parameter lower bound for latency, but among learned forecasters DLinear is the lowest-cost option. PatchTST-lite increases latency and memory for better stress accuracy, while Official PatchTST, native iTransformer, and official TimeMixer are heavier references. This cost table is important because AIOps-style workflows often contain both low-latency monitoring paths and slower offline planning paths. GPU memory is measured by the per-run profiler and rounded to MB; 1 MB-level TimeMixer differences across clean, stress, and aggregate rows reflect measurement aggregation variance rather than configuration changes.

The main capacity-related evidence comes from Alibaba and Salesforce/Borg because they are resource-level telemetry sources. A model that is attractive for one stress or dataset can still be a poor choice under another operational constraint; the meaningful comparison is within the same dataset, stress, and decision objective.

Table ?? gives the Alibaba detail corresponding to the cross-source summary in Table ?. Winners are selected only from the core learned comparable pool, excluding LastValue, Chronos-Bolt, and LTSF-bridge references. This single-run Alibaba summary includes official/native-wrapper models for coverage, while Table ?? separately checks the lightweight core pool over five seeds on both multivariate sources. DLinear is the low-cost learned baseline, while patch-based or official models can become better candidates when robustness or the decision proxy is prioritized.

Salesforce/Borg results reproduce the high-level lesson on a second multivariate telemetry source. This supports the benchmark/system-evaluation framing: deployment constraints and time-series behavior jointly determine the right forecaster.

Table ?? provides the decision-proxy evaluation on the two multivariate telemetry sources. Reactive HPA is reported

TABLE XI
FORECAST-TO-CAPACITY OPEN-LOOP DECISION PROXY ON MULTIVARIATE TELEMETRY SOURCES. REACTIVE HPA IS REPORTED AS A CONTROLLER REFERENCE AND IS EXCLUDED FROM FORECASTING-POLICY WINNER SELECTION.

Source	Model	Cost	Under	Over	Peak miss	P95 ms
Alibaba	Reactive HPA	0.3423	0.0317	0.1838	0.0091	controller ref.
Alibaba	PatchTST-lite	0.4136	0.0426	0.2005	0.7179	0.6995
Alibaba	DLinear	0.4175	0.0384	0.2257	0.6951	0.1673
Alibaba	RACE-DLinear	0.4225	0.0426	0.2094	0.7254	0.3935
Salesforce/Borg	Reactive HPA	0.1591	0.0059	0.1298	0.0026	controller ref.
Salesforce/Borg	PatchTST-lite	0.2449	0.0111	0.1894	0.2	0.6435
Salesforce/Borg	RACE-DLinear	0.2808	0.0303	0.1293	0.4851	0.3731
Salesforce/Borg	DLinear	0.2813	0.0188	0.1876	0.3366	0.1497

TABLE XII
CAPACITY-PROXY SENSITIVITY FOR METRIC-CHANNEL OUTAGE. WINNERS MINIMIZE TOTAL NORMALIZED DECISION COST WITHIN EACH SOURCE AND SETTING.

Sensitivity	Alibaba winner	Alibaba cost	Salesforce/Borg winner	Salesforce/Borg cost	Pattern
Cost 2:1	PatchTST-lite	0.2993	PatchTST-lite	0.2933	same
Cost 5:1	PatchTST-lite	0.4509	PatchTST-lite	0.3515	same
Cost 10:1	PatchTST-lite	0.7036	PatchTST-lite	0.4485	same
Horizon 96	PatchTST-lite	0.5278	PatchTST-lite	0.4013	same

separately as a controller reference because it reacts after each realized step; in this short-horizon proxy without closed-loop scaling delay, cold starts, queueing, scheduler constraints, service dependencies, or service-level SLOs, such a reactive controller can achieve low total normalized cost. Among forecasting policies, Alibaba and Salesforce/Borg show different tradeoffs: DLinear is a low-latency planning baseline, while PatchTST-lite is more attractive on Salesforce/Borg and under metric-outage conditions. The decision proxy is used to compare forecasting-model choices under a fixed asymmetric-cost protocol, while Reactive HPA helps separate controller-style reaction from proactive forecasting.

These rows reinforce the same message as Table ??: the best clean forecaster, the best robust forecaster, and the best decision-proxy policy need not be the same learned model.

Table ?? adds the compact sensitivity check requested by the capacity-proxy claim. Under metric-channel outage, PatchTST-lite remains the lowest total-cost forecasting policy on both multivariate sources at 2:1, 5:1, and 10:1 under/over cost ratios, and also at prediction horizon 96 in the lightweight horizon sweep. This does not make PatchTST-lite a universal winner: clean and delayed settings can change with the cost ratio, and DLinear remains the low-latency learned model. It does show that the metric-outage decision-proxy conclusion is not an artifact of one default 5:1 setting. The evidence for Claim 3 is Table ??, Table ??, Table ??, and Table ??.

E. Claim 3b: Deployment Constraints Become Selection Rules

Takeaway: benchmark evidence can be translated into conservative model-selection rules; the policy example is illustrative, not an optimized router.

The benchmark is intended to support model-selection decisions rather than only rank forecasting models. Conservative rules from the evidence are simple: use DLinear when P95 latency dominates, consider patch-based models when metric-channel outage or batch planning is more important, and avoid

TABLE XIII

FIVE-SEED WINNER STABILITY OVER ALL 10 CORE SOURCE-STRESS SETTINGS IN THE LIGHTWEIGHT POOL. ROWS COMPARE THE BEST-MSE, BEST DECISION-COST, AND LOWEST-P95 LEARNED MODELS; P-VALUES ARE PAIRED TESTS OVER DECISION COST FOR MSE-VS-DECISION FLIPS.

Source	Stress	Seeds	Best MSE	Best decision	Low P95	MSE != decision	p-value
Alibaba	Clean	5	RACE-DLinear	RACE-DLinear	DLinear	N	—
Alibaba	Delayed tail	5	RACE-DLinear	RACE-DLinear	DLinear	N	—
Alibaba	Level shift	5	RACE-DLinear	RACE-DLinear	DLinear	N	—
Alibaba	Missing points	5	RACE-DLinear	PatchTST-lite	DLinear	Y	0.0011
Alibaba	Metric outage	5	PatchTST-lite	PatchTST-lite	DLinear	N	—
Salesforce/Borg	Clean	5	DLinear	RACE-DLinear	DLinear	Y	0.0007
Salesforce/Borg	Delayed tail	5	DLinear	RACE-DLinear	DLinear	Y	0.0037
Salesforce/Borg	Level shift	5	DLinear	RACE-DLinear	DLinear	Y	0.0064
Salesforce/Borg	Missing points	5	PatchTST-lite	PatchTST-lite	DLinear	N	—
Salesforce/Borg	Metric outage	5	PatchTST-lite	PatchTST-lite	DLinear	N	—

treating any single clean-MSE row as a universal winner. StressRoute is not proposed as a new routing algorithm; it is an auditable decision-support example showing how validation evidence can be converted into conservative model-selection rules under telemetry-health and latency constraints. Because its oracle gap remains non-negligible, StressRoute should be interpreted as a benchmark component and practitioner illustration rather than as an optimized deployment policy.

F. Stability Across Sources

Takeaway: the exact winning model changes by dataset, but the higher-level deployment lesson is stable across the two multivariate telemetry sources.

Fixed stress points are useful, but an offline deployment-stress benchmark should also test whether conclusions hold as corruption becomes stronger. We therefore add severity curves for point missingness, variable outage, delayed telemetry, and sensor noise on Alibaba and Salesforce/Borg. Figure ?? summarizes the main visual evidence. A robust operational model should not only have a low clean MSE; its degradation should be gradual as telemetry quality worsens. The Salesforce/Borg rows are especially useful because they show the same structural tradeoff on a second multivariate operational source: DLinear remains cheap and stable on clean/delayed cases, while PatchTST-lite is more robust under point and metric missingness.

To reduce the chance that model ranking is an artifact of one random seed, we also run Alibaba and Salesforce/Borg core scenarios with five seeds. Table ?? reports all 10 core source-stress settings for the lightweight pool. The table separates non-trivial MSE-vs-decision-cost reversals from structural accuracy-vs-latency differences. In the core lightweight pool, MSE and decision-cost winners differ in 4/10 source-stress settings; paired tests over decision cost are reported as supporting statistics rather than as the primary stability claim. Separately, DLinear remains the low-latency learned model in all 10 settings.

G. Baseline Scope

Takeaway: the lightweight robust baseline is useful for stress analysis, while the main contribution is the benchmark and deployment evidence chain.

The ablation clarifies the role of RACE-DLinear in the benchmark. Removing stress augmentation causes a large

degradation on GAIA missing-30, indicating that exposure to deployment-style corruption is more important than the extra model wrapper itself. Masking and calibration affect MAE and capacity cost, but these differences are better interpreted as benchmark evidence about stress-aware training than as a separate method contribution.

This supports the final positioning of the work: the contribution is the controlled stress-to-decision evaluation pipeline and the resulting accuracy–robustness–latency analysis. RACE-DLinear is useful because it tests whether a low-cost robust variant can close part of the gap, while the central evidence remains the benchmark-driven model-selection analysis across stress, latency, and decision cost.

IX. CASE STUDIES

The case studies highlight representative failure mechanisms behind the aggregate results. The selected windows cover four patterns: NetMan telemetry outage removes the only observable KPI channel, Salesforce/Borg delayed telemetry turns forecasting into stale-context extrapolation, NetMan point missingness exposes an accuracy-latency tradeoff, and Alibaba metric outage stresses cross-metric context. In each case, the clean window hides the degradation mechanism, while Tables ??–?? and Figure ?? provide the aggregate evidence.

X. THREATS TO VALIDITY

Data scope and realism. The resource-capacity evidence in AIOpsStressBench is grounded in public operational telemetry at machine or resource-proxy level. Alibaba 2018 provides machine-level resource traces, and the Salesforce/Borg public subset provides multivariate operational telemetry with one all-zero dynamic metric retained for schema transparency. GAIA and NetMan broaden KPI stress diversity and failure analysis, while Alibaba and Salesforce/Borg carry the main multivariate resource-telemetry evidence for latency, stress, and decision-proxy analysis. A stronger future extension would add reproducible public service-level request or SLO traces.

Decision-proxy assumptions. The forecast-to-capacity evaluator is designed as a reproducible open-loop proxy for within-protocol comparison of forecasting-policy consequences. It captures under-provision area, over-provision area, peak misses, and headroom sensitivity under a fixed asymmetric-cost rule, while abstracting away queueing, scheduler constraints, cold starts, closed-loop scaling delay, service dependencies, and service-level SLOs. Reactive HPA therefore serves as a controller reference inside this proxy setting rather than as a full control-system conclusion.

Baseline comparability. Official PatchTST, native iTransformer, and official TimeMixer are integrated into the native stress pipeline for Alibaba and Salesforce/Borg core scenarios, while Chronos-Bolt is included as a bounded zero-shot reference over a subset of windows. Older LTSF CSV bridge runs remain reference-only because the bridge flattens entity-metric channels and is not fully equivalent to the native per-entity target protocol. The benchmark therefore separates

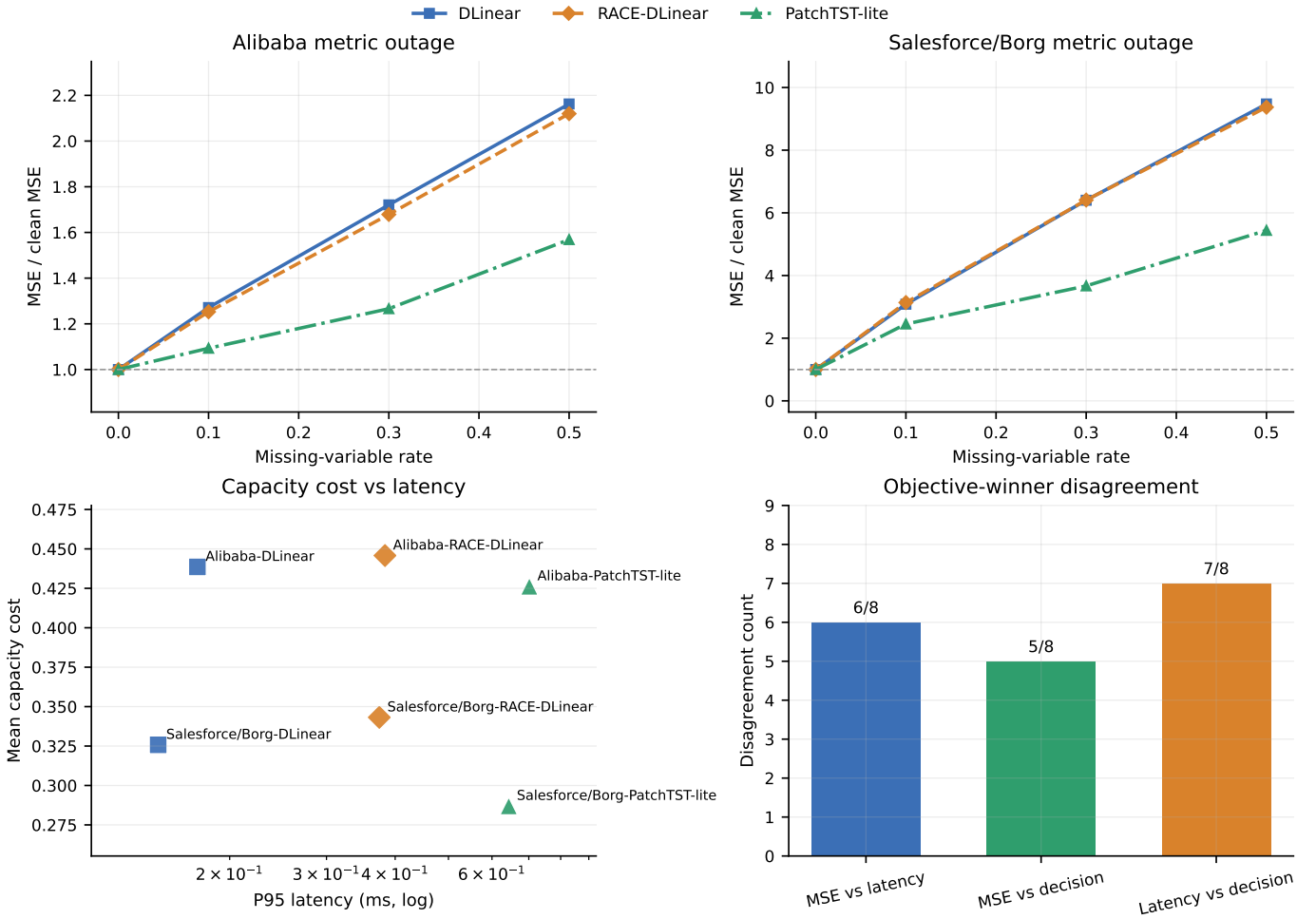


Fig. 2. Telemetry degradation changes model-selection tradeoffs across both multivariate sources. Panels show (a) Alibaba metric outage, (b) Salesforce/Borg metric outage, (c) decision cost versus latency, and (d) objective-winner disagreement counts over the same eight compact rows in Table ???. The figure supports within-protocol model-selection analysis under controlled offline stress.

lightweight repeated comparisons, official-wrapper coverage comparisons, and bounded reference runs rather than treating every backbone as the same experimental object.

Claim scope. RACE-DLinear is used as a lightweight robust baseline for stress analysis, while the central contribution of the paper is the benchmark protocol, model-selection analysis, deployment-cost evaluation, and case-study evidence. The current empirical scope focuses on reproducible stress-to-decision evidence over public operational telemetry rather than on exhaustive foundation-model coverage or a standalone method paper.

XI. REPRODUCIBILITY AND ARTIFACT

Artifact:

<https://anonymous.4open.science/r/AIOpsStressBench-anon-artifact-4EE5/>. The artifact contains data conversion scripts, released NPZ tensors and manifests, stress operators, model wrappers, and the scripts used to generate the main tables and figures.

The artifact entry point is `ANON_ARTIFACT.md`, which documents the benchmark manifest, data-processing steps, experiment notes, and table/figure provenance. Repository identifiers are anonymized during review and will be deanonymized after acceptance without changing the released experimental contents.

A minimal reviewer-verification path uses the public NPZ tensors, input length 96, prediction horizon 24, chronological 65/15/20 splits, and seed 42 to reproduce the core winner and latency results from the main paper. The artifact entry point documents the exact commands for the minimal and extended paths. Extended runs reproduce severity curves, multi-seed checks, capacity sensitivity, official-wrapper comparisons, and the bounded Chronos-Bolt reference. This separation keeps the main evidence chain auditable without requiring reviewers to rerun the full experimental sweep.

XII. CONCLUSION

This paper introduced AIOpsStressBench, a reproducible of-fline stress-to-decision benchmark for operational time-series

forecasting. The benchmark combines public telemetry, controlled telemetry-stress operators, latency and memory profiling, severity analysis, and a forecast-to-capacity open-loop decision proxy. Under this controlled protocol, clean-MSE selection can differ from the model selected under telemetry degradation, latency budgets, or asymmetric decision costs. This does not imply that clean forecasting benchmarks are uninformative; rather, it shows that they are incomplete for AIOps-style settings where telemetry health and deployment constraints are part of the model-selection problem. Robustness is also stress-family specific: patch-based context can help under metric outage, but this advantage is not universal across noise, delay, and latency-constrained settings. AIOpsStress-Bench provides a reproducible stress-to-decision evaluation layer for operational forecasting over public telemetry. Future extensions should add public service-level traces and closed-loop control simulation.

REFERENCES

- [1] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *ICLR*, 2023.
- [2] Y. Liu et al., "iTransformer: Inverted transformers are effective for time series forecasting," in *ICLR*, 2024.
- [3] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *AAAI*, 2023.
- [4] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *ICLR*, 2023.
- [5] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.
- [6] R. Godahewa, C. Bergmeir, G. I. Webb, R. J. Hyndman, and P. Montero-Manso, "Monash time series forecasting archive," in *NeurIPS Datasets and Benchmarks*, 2021.
- [7] T. Aksu, G. Woo, J. Liu, X. Liu, C. Liu, S. Savarese, C. Xiong, and D. Sahoo, "GIFT-Eval: A benchmark for general time series forecasting model evaluation," arXiv:2410.10393, 2024.
- [8] O. Shchur, A. F. Ansari, C. Turkmen, L. Stella, N. Erickson, P. Guerron, M. Bohlke-Schneider, and Y. Wang, "fev-bench: A realistic benchmark for time series forecasting," arXiv:2509.26468, 2025.
- [9] Alibaba Cluster Trace Program, "Alibaba cluster trace 2018," 2018.
- [10] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Zhang, and J. Zhou, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *ICLR*, 2024.
- [11] A. F. Ansari et al., "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024.
- [12] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *ICML*, 2024.
- [13] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *ICML*, 2024.
- [14] L. Guibert, B. Pasquier, F. Montet, and B. Wolf, "Benchmarking time series foundation models on their accuracy and energy consumption," in *Proceedings of the Swiss AI Days*, PMLR 309, 2026.
- [15] G. Woo, C. Liu, A. Kumar, and D. Sahoo, "Pushing the limits of pre-training for time series forecasting in the CloudOps domain," arXiv:2310.05063, 2023.
- [16] B. Cohen et al., "This time is different: An observability perspective on time series foundation models," arXiv:2505.14766, 2025.
- [17] K. Rzacca et al., "Autopilot: Workload autoscaling at Google scale," in *EuroSys*, 2020.
- [18] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: ML-based and QoS-aware resource management for cloud microservices," in *ASPLOS*, 2021.
- [19] Z. Zhou, C. Zhang, L. Ma, J. Gu, H. Qian, Q. Wen, L. Sun, P. Li, and Z. Tang, "AHPA: Adaptive horizontal pod autoscaling systems on Alibaba Cloud Container Service for Kubernetes," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 13, pp. 15621–15629, 2023.
- [20] Google Cloud, "Behavior of metric-based alerting policies: Missing data," Cloud Monitoring documentation.
- [21] Google Cloud, "Metric latency and data freshness," Cloud Monitoring documentation.
- [22] Prometheus Authors, "Staleness," Prometheus documentation.
- [23] Kubernetes Authors, "Horizontal Pod Autoscaling," Kubernetes documentation.
- [24] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific Reports*, vol. 8, 2018.
- [25] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, "BRITS: Bidirectional recurrent imputation for time series," in *NeurIPS*, 2018.
- [26] Y.-F. Zhang, Q. Wen, X. Wang, W. Chen, L. Sun, Z. Zhang, L. Wang, R. Jin, and T. Tan, "OneNet: Enhancing time series forecasting models under concept drift by online ensembling," in *NeurIPS*, 2023.
- [27] J.-C. Gagnon-Audet, K. Ahuja, M.-J. Darvishi-Bayazi, P. Mousavi, G. Dumas, and I. Rish, "WOODS: Benchmarks for out-of-distribution generalization in time series," *Transactions on Machine Learning Research*, 2023.
- [28] CloudWise-OpenSource, "GAIA-DataSet: Generic AIOps Atlas," GitHub repository, 2021.
- [29] Z. Li, N. Zhao, S. Zhang, Y. Sun, P. Chen, X. Wen, M. Ma, and D. Pei, "Constructing large-scale real-world benchmark datasets for AIOps," arXiv:2208.03938, 2022.